CHRONO SUPPORT FOR HUMAN-IN-THE-LOOP SIMULATION

Speaker: Kyle Sha
(Simulation-Based Engineering Lab)

Overview of the human-in-the-loop tutorial

- Human-in-the-loop = the simulation runs at wall-clock speed and a person closes the control loop
- In this tutorial, you will learn how to
  - PART 1: Keep a Chrono::Vehicle simulation real-time and measure how well you did
  - PART 2: Put a human on the keyboard with ChInteractiveDriver
  - PART 4: Bring a device Chrono doesn't know about, and close the loop back to it
  - Then, briefly: PARTS 6-8 - change gear, drive the Mcity digital twin, steer something that is not a car
  - Parts 3 and 5 (gamepad/wheel, vehicle picker, overlay flags) are in the repo - no time today
- Everything runs in PyChrono; no special hardware is required
- The whole human-in-the-loop contract is: spin once per step, three floats in, state out

Before we start ……

- This is a walkthrough - nothing to install right now; everything runs on my machine
- Code, slides and a written walkthrough: github.com/ksha23/chrono-hil-tutorial
- To run it yourself afterward:
  - `conda create -n chrono_hil -c projectchrono -c conda-forge pychrono pygame python=3.13`
  - `conda activate chrono_hil`
- Do NOT `pip install pychrono` - the PyPI package with that name is an unrelated library
- python tutorial_HIL_driver.py opens an Irrlicht window with an HMMWV - drive it with the arrow keys
- All switches are in the CONFIGURATION section at the bottom of tutorial_HIL_driver.py

Part 1: Keep the simulation real-time

What "real-time" means in Chrono

- Chrono integrates as fast as it can; it has no notion of wall-clock time by itself
- Real Time Factor: RTF = compute time for a step / step size
  - RTF < 1: faster than real time - we must WAIT after each step
  - RTF > 1: slower than real time - nothing can be done except a cheaper model or a bigger step
- Soft real-time: spin after each step until wall time catches up with simulation time
  - No OS scheduling guarantees; good enough for a driving simulator, not for a control ECU
- Three equivalent mechanisms in PyChrono (next slides): `ChRealtimeStepTimer`, `ChVehicle.EnableRealtime`, or your own

Step 1: The simulation loop (Lines 674 to 725)

```
while vis.Run():
    sim_time = system.GetChTime()

    # Render scene
    if step_number % render_steps == 0:
        follow_camera()
        vis.BeginScene()
        vis.Render()
        vis.EndScene()

    # PART 4 samples the device here; PART 6 applies any gear command
    ...

    # Get driver inputs (three floats) - the whole human-in-the-loop contract
    driver_inputs = driver.GetInputs()

    # PART 8: hand the three numbers to whatever is being controlled
    plant.apply(driver_inputs)

    # Update modules (process inputs from other modules)
    driver.Synchronize(sim_time)
    terrain.Synchronize(sim_time)
    plant.synchronize(sim_time, driver_inputs)
    vis.Synchronize(sim_time, driver_inputs)

    # Advance simulation for one timestep for all modules
    driver.Advance(step_size)
    terrain.Advance(step_size)
    plant.advance(step_size)          # spins here if REALTIME == "vehicle"
    system.DoStepDynamics(step_size)  # only for a plant that does not step itself
    vis.Advance(step_size)
```

Synchronize: every module reads what it needs from the others at time t. Advance: every module integrates from t to t + step
The driver is queried BEFORE Synchronize - inputs are sampled once and held for the step
Elided: the device/gear block (Parts 4 and 6), and the guards that let a plant run with no terrain and no vehicle camera (Part 8)

Step 2: Measure it - sim time vs wall time (Lines 727 to 743)

```
# Console report: how far is the sim from wall time?
if step_number % report_steps == 0:
    wall = time.perf_counter() - wall_start
    # A ChVehicle measures its own real-time factor per step. The rover
    # and the crane do not, so it is measured here over the reporting
    # interval instead: same quantity, coarser sampling.
    if vehicle:
        rtf = vehicle.GetRTF()
    else:
        d_wall = wall - last_wall
        d_sim = sim_time - last_sim
        rtf = (d_wall / d_sim) if d_sim > 0 else 0.0
        last_wall, last_sim = wall, sim_time
    print(f"{sim_time:7.2f} {wall:7.2f} {wall - sim_time:+7.3f} {rtf:6.2f}"
          f" {plant.speed():7.2f}  {driver_inputs.m_steering:6.2f}"
          f" {driver_inputs.m_throttle:5.2f} {driver_inputs.m_braking:5.2f}"
          f"  {plant.status()}")
```

drift = wall time - simulation time: negative means the sim is ahead of the clock

`vehicle.GetRTF()` is the true RTF of the last step, even when real time is enforced

PART 8: a rover or a crane has no `GetRTF()`, so the same number is measured here over the reporting interval

Show Time

- REALTIME = "none": watch the drift column run away
- Change step_size to 5e-4 - RTF goes above 1 and the sim can never be real-time

Step 3: Enforce real time, three ways (Lines 630 to 645)

```
# ------------------------------------------------------------------------
# Real-time setup (PART 1)
# ------------------------------------------------------------------------
realtime_mode = REALTIME
if REALTIME == "vehicle":
    if vehicle:
        vehicle.EnableRealtime(True)
    else:
        # PART 8: there is no vehicle to do the spinning, so fall back to
        # the timer that does the same thing from outside.
        realtime_mode = "per_step"
rt_timer = chrono.ChRealtimeStepTimer()  # used when REALTIME == "per_step"
cum_timer = CumulativeRealtimeTimer()    # used when REALTIME == "cumulative"
```

"vehicle": ChVehicle::EnableRealtime(True) spins inside vehicle.Advance() using a ChRealtimeStepTimer
"per_step": the same timer, called explicitly at the end of the loop (next slide)
"cumulative": our own timer that compares total simulated time to total wall time
PART 8: a rover or a crane has no vehicle to spin for it, so "vehicle" quietly becomes "per_step" - same policy

Step 4: A drift-free timer in 10 lines (Lines 71 to 88)

```
class CumulativeRealtimeTimer:
    """Soft real-time that does not accumulate drift.

    ChRealtimeStepTimer measures each step independently: any time lost on a
    slow step (or spent in Python between steps) is never recovered.  This timer
    instead compares the *total* simulated time with the *total* wall time, so a
    slow step is followed by a few fast steps that catch back up.
    """

    def __init__(self):
        self.reset()

    def reset(self):
        self.t_start = time.perf_counter()

    def spin(self, sim_time):
        while time.perf_counter() - self.t_start < sim_time:
            pass  # spin in place until real time catches up
```

Compare TOTAL simulated time with TOTAL wall time - lost time is caught up on the next fast steps
This is how driving simulators (e.g. Chrono::HIL) keep their clock; the same idea also works across processes

Show Time

- Measured on a laptop, HMMWV + TMeasy tires, step 3 ms (RTF ~ 0.14):
  - "none": drift -69 s after 17 s of wall time
  - "per_step": drift +0.012 s after 17 s and growing
  - "vehicle": drift +0.021 s after 17 s and growing
  - "cumulative": drift +0.000 s
- Drift matters more the closer you are to RTF = 1

Part 2: A human on the keyboard

Step 1: ChInteractiveDriver (Lines 535 to 546)

```
elif INPUT_SOURCE == "keyboard":
    ### PART 2: keyboard through the Irrlicht window ###
    # Arrow keys steer/accelerate/brake, 'C' centers steering, 'R' releases
    # the pedals, 'L' locks the current inputs.
    driver = veh.ChInteractiveDriver(vehicle)
    steering_time = 1.0  # time to go from 0 to +1 (or from 0 to -1)
    throttle_time = 1.0  # time to go from 0 to +1
    braking_time = 0.3   # time to go from 0 to +1
    driver.SetSteeringDelta(render_step_size / steering_time)
    driver.SetThrottleDelta(render_step_size / throttle_time)
    driver.SetBrakingDelta(render_step_size / braking_time)
    driver.SetGains(4.0, 4.0, 4.0)  # first-order lag from key target to applied input
```

Arrow keys steer/accelerate/brake, C centers steering, R releases the pedals, L locks the inputs
Each keypress moves a TARGET by "delta"; the applied input follows the target with a first-order lag (SetGains)
delta = render_step / time-to-full-scale: full steering lock takes 1 s, full braking 0.3 s

Step 2: Route the window events to the driver (Lines 589 to 592)

```
vis.AddSkyBox()
plant.attach(vis)
if INPUT_SOURCE in ("keyboard", "gamepad"):
    vis.AttachDriver(driver)  # route Irrlicht key/joystick events to the driver
```

The Irrlicht visual system owns the keyboard; AttachDriver forwards key events to the driver
Nothing else in the loop changes - the driver still just returns three floats
The same driver reads a gamepad or wheel natively - SetInputMode(JOYSTICK), presets ship with Chrono
PART 6: this one call also binds Chrono's own gear keys - Z, X, T, [and] - without a line of your code

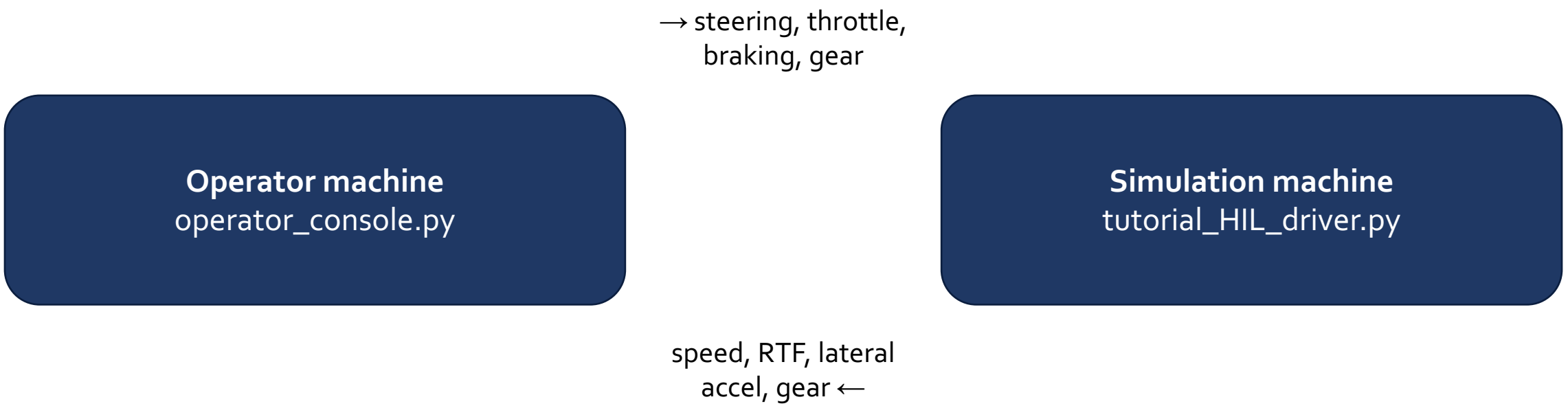
Show Time

- Driving the HMMWV with the arrow keys - watch the Irrlicht HUD (speed, inputs, RTF)

Part 4: A device Chrono doesn't know about, and closing the loop

a UDP console, a gamepad, a steering wheel, a phone, another process…

Part 4's architecture: two processes, one UDP contract



Both directions are plain UDP datagrams - two processes, possibly two machines, no shared memory. The grey arrow only fires when SEND_FEEDBACK = True. PART 6 adds the gear as a fourth field, in both directions.

The human-in-the-loop contract

- A driver is just three floats per step: steering [-1, 1], throttle [0, 1], braking [0, 1]
- `veh.ChDriver` is the plain container: `SetSteering()`, `SetThrottle()`, `SetBraking()`
- So the recipe for ANY device is:
  - 1. poll the device (never block the physics loop)
  - 2. rate-limit / smooth (a human arm is not a step function)
  - 3. write into the ChDriver once per step - zero-order hold until the next step
- Today: a UDP operator console in a second terminal - the operator side has no Chrono dependency at all

Step 1: A UDP input source (Lines 219 to 269)

```
class UdpInput:
    """Receive 'steering,throttle,braking[,gear]' datagrams from an operator.
    Non-blocking: the physics loop never waits.  No packet since the last step
    means the previous target is held (zero-order hold)."""

    def __init__(self, port=9870):
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.sock.bind(("0.0.0.0", port))
        self.sock.setblocking(False)
        self.operator_addr = None
        self.last = (0.0, 0.0, 0.0)
        self.gear_commands = []

    def poll(self):
        # Drain everything queued and keep only the newest packet.  Gear commands
        # are the exception: they accumulate, because dropping one loses a shift.
        while True:
            try:
                data, addr = self.sock.recvfrom(256)
            except BlockingIOError:
                break
            fields = data.decode().split(",")
            if len(fields) < 3:
                continue
            try:
                s, t, b = (float(x) for x in fields[:3])
            except ValueError:
                continue
            self.last = (s, t, b)
            self.operator_addr = addr
            if len(fields) > 3 and fields[3] and fields[3] != "-":   # PART 6
                self.gear_commands.append(fields[3][0])
        return self.last
```

Non-blocking socket, drained every step; the newest packet wins, an empty queue holds the last value

The operator can be anywhere on the network - and the console has no Chrono dependency at all

PART 6 added the optional 4th field. Keeping it optional is what stops an older console from breaking: 3 fields still parse

Step 2: Sample once per step, then hold (Lines 684 to 701)

```
# PART 4: sample the device ONCE per step and hold it for the step
if device is not None:
    smoother.set_target(*device.poll())
    smoother.advance(step_size)
    smoother.apply_to(driver)
    # PART 6: gear commands are events, not levels, so they are applied
    # once each rather than held like the three numbers above.
    if gearbox is not None:
        for command in device.take_gear_commands():
            gearbox.command_char(command)

# Get driver inputs (three floats) - this is the whole human-in-the-loop contract
driver_inputs = driver.GetInputs()

# PART 8: hand the three numbers to whatever is being controlled
plant.apply(driver_inputs)
```

poll -> smooth -> write, then GetInputs() as before; the physics loop never waits on the device
The three numbers are held; the gear commands are drained. One press of ']' is one upshift, not "keep upshifting"

Step 3: Send state back at render rate (Lines 745 to 755)

```
# PART 4: feedback to the operator (speed, RTF, lateral acceleration, applied
# inputs, and since PART 6 the gear)
if SEND_FEEDBACK and device is not None and step_number % render_steps == 0:
    acc = (vehicle.GetPointAcceleration(chrono.ChVector3d(0, 0, 0)).y
           if vehicle else 0.0)
    rtf = vehicle.GetRTF() if vehicle else 0.0
    gear = gearbox.describe() if gearbox else plant.status() or "--"
    device.send_feedback(
        f"{sim_time:.3f},{plant.speed():.3f},{rtf:.3f},{acc:.3f},"
        f"{driver_inputs.m_steering:.3f},{driver_inputs.m_throttle:.3f},"
        f"{driver_inputs.m_braking:.3f},{gear}")
```

Speed, RTF and lateral acceleration go back to whoever sent us inputs (UdpInput remembers the address)
Send at render rate (50 Hz), not at every physics step - the human cannot use 333 Hz
Whatever the plant is, the packet has the same shape: the console renders a crane's swing angle the same way it renders a gear

Show Time

- Two terminals on one laptop over localhost - or two machines over the network, same code either way
- SEND_FEEDBACK = True: speed, RTF and lateral acceleration come back to the console while you drive
- Stop the console mid-drive: the last input is held - what should a real simulator do here?

Parts 6-8: the same loop, three more things

changing gear, driving somewhere real, and steering something that is not a car

Part 6: Change gear - and why it is not a fourth float

```
# A ChDriver carries three numbers, and a gear is not one of them: the
# gearbox belongs to the VEHICLE, so it is reached through the vehicle.

transmission = vehicle.GetTransmission()
transmission.ShiftUp()                        # or ShiftDown(), SetGear(n)
transmission.asAutomatic().SetDriveMode(...)  # D / N / R
transmission.asAutomatic().SetShiftMode(...)  # let it shift, or row it

# On the keyboard you write none of this - vis.AttachDriver(driver) binds:
#     Z  D <-> R     X  neutral     T  AUTO <-> MANUAL     [ ]  down / up
#
# For a device Chrono has never heard of, nothing is wired up. That is the
# point of PART 4: hil_gearbox.Gearbox turns one character off the socket
# into the calls above, and operator_console.py sends it in a 4th field.
```

A steering value is a LEVEL: re-sent every frame, and a lost packet costs nothing
A gear change is an EVENT: a lost one is a shift that never happened - so it is sent once, on the key-down edge, and applied exactly once
VEHICLE = "audi" is the only model here with a manual gearbox: it is assembled from JSON, not from a model wrapper class, and the wrappers each hard-code one powertrain

Part 7: Drive somewhere real - the Mcity digital twin

- `SCENE = "mcity"` swaps the 200 x 200 m patch for a real 32-acre test facility
  - road surface driven as a collision mesh; buildings, poles, signal heads drawn from a placement manifest
  - `MCITY_DETAIL` = "ground" | "light" | "full" trades scenery for frame rate - start at "ground" on a laptop
  - 860 placements cost 230 meshes: one ChVisualShape, added to a body many times, each time with its own frame
- Third-party and generated, not shipped: a missing dataset falls back to flat terrain with instructions
- Two things that cost time to rediscover:
  - Drive the DRAWN geometry: Mcity's OpenDRIVE elevation differs from the road mesh by -0.24 to +0.29 m (5th/95th pct)
  - Read the spawn height from the mesh: `RigidTerrain` raycasts a collision system that does not exist yet and answers 0, which on a 274 m datum drops the car out of the world

Part 8: Control something that isn't a car

- `PLANT` switches what the same three numbers drive. The devices and the loop do not change
  - `"vehicle"` a Chrono::Vehicle, as in Parts 1-7
  - `"rover"` a Viper rover: steering -> wheel angle, throttle -> commanded wheel speed
  - `"crane"` a gantry crane: steering -> cross-travel, throttle/braking -> forward/reverse travel
- The crane has no Chrono::Vehicle at all: four bodies, two motors, a distance constraint
  - the payload swings with nothing to damp it but the operator - which is the whole point
- No ChVehicle means no ChInteractiveDriver: both take input over UDP, console unchanged
  - `DriverInputs` is the whole ChDriver contract in twelve lines of Python: three floats, three setters, GetInputs()

Show Time

- `PLANT = "crane"` over udp: put the load on the green pad with the swing under 2 degrees
  - the console prints swing angle and miss distance - same telemetry path, same packet shape as the car
- `VEHICLE = "audi"`, `TRANSMISSION = "manual"`: pull away in third and feel the engine bog down
- `SCENE = "mcity"` with `MCITY_DETAIL = "ground"`: the same three numbers, somewhere real
- `PLANT = "rover"` and hold the brake: it coasts rather than stopping - ViperDriver has no brake input

Where this goes next

- Chrono::HIL (github.com/zzhou292/chrono-HIL): the same three-float interface, scaled up
  - SDL steering wheels with force feedback, cumulative real-time timer, multi-vehicle traffic
  - ROS 2 and Unity bridges, teleoperation over the network, deformable (SCM) terrain
- Chrono::Sensor: cameras/lidar rendered from the vehicle for the operator's screen
- Chrono::Synchrono: several humans in the same world, each on their own machine
- Takeaways
  - Real time in Chrono is soft: spin once per step, and measure the drift
  - The human interface is three floats per step, sampled once and smoothed
  - Never let the input device block the physics loop
  - Levels are held, events are drained - a gear is an event
  - Nothing in the pattern needs a vehicle: swap the plant, keep the loop

Any questions?

Thank you.

kasha2@wisc.edu
Simulation-Based Engineering Lab
University of Wisconsin - Madison

Lab website: http://sbel.wisc.edu
Chrono website: http://www.projectchrono.org
Source code: https://github.com/projectchrono
Tutorial code + slides: https://github.com/ksha23/chrono-hil-tutorial